Ref. [42] — Adaptive Subgradient Methods for Online Learning and Stochastic Optimization

J. Duchi, E. Hazan, Y. Singer | Journal of Machine Learning Research 12, 2121–2159 (2011)

1. Problem: one learning rate is not right for every feature

In sparse learning, a useful feature might appear in only a few training examples, while other coordinates receive gradients all the time. Ordinary SGD treats all coordinates with the same nominal learning rate. Duchi et al. introduce a family of proximal and adaptive subgradient methods that uses *past gradient geometry* to adjust each coordinate's effective step. Their central theoretical framework is online *convex* optimization and regret, not non-convex saddle-point escape.

2. First principles: coordinate-wise scaling

For a gradient sequence $g_t \in \mathbb{R}^d$, accumulate squared gradients for coordinate j:

$$G_{t,j} = \sum_{s=1}^{t} g_{s,j}^2.$$

The familiar diagonal AdaGrad update (ignoring optional projection or composite regularization) is

$$\boxed{x_{t+1,j} = x_{t,j} - \frac{\eta\, g_{t,j}}{\sqrt{G_{t,j}} + \delta}, \quad \delta > 0.}$$

A feature with repeated large gradients accumulates a large denominator and receives smaller steps. If a coordinate has received few gradients, its effective step stays comparatively large. The δ is a numerical stabilizer; the paper develops adaptive *proximal* updates more generally, including full-matrix and diagonal choices.

3. Where does this update come from?

A quadratic regularizer determines what "distance" the optimizer considers costly. At iteration t, imagine minimizing a linear approximation to the current loss, while penalizing the move in an adaptive metric $H_t \succ 0$:

$$x_{t+1} = \arg\min_x \left\{ g_t^\top (x - x_t) + \frac{1}{2\eta}(x - x_t)^\top H_t (x - x_t) \right\}.$$

Differentiate this *teaching proximal surrogate* with respect to x:

$$g_t + \frac{1}{\eta} H_t (x_{t+1} - x_t) = 0 \quad \Longrightarrow \quad x_{t+1} = x_t - \eta H_t^{-1} g_t.$$

Choosing H_t diagonal, with entries related to $\sqrt{\sum_{s \le t} g_{s,j}^2}$, gives the coordinate-wise AdaGrad intuition. For constrained or regularized problems, replace the explicit update by the appropriate proximal step.

4. The paper's formal contribution: adaptive regret

In online convex learning, the player selects x_t and suffers $f_t(x_t)$. Compare cumulative loss with the best fixed feasible point chosen in hindsight:

$$R_T = \sum_{t=1}^{T} f_t(x_t) - \min_{x \in \mathcal{X}} \sum_{t=1}^{T} f_t(x).$$

The authors give regret bounds that adapt to observed gradient sequences and compete with favorable hindsight choices of proximal geometry. An illustrative diagonal form includes a term of order

$$\boxed{R_T \;\lesssim\; D_\infty \sum_{j=1}^{d} \sqrt{\sum_{t=1}^{T} g_{t,j}^2}}$$

with problem-specific domain constants and tuning, rather than simply $O(\sqrt{dT})$ in a worst-case isotropic bound. This displayed expression is a schematic consequence under appropriate bounded-domain assumptions, not a verbatim complete theorem with all constants.

Main result. Coordinate-wise and full-matrix adaptive proximal methods exploit the geometry of earlier gradients and enjoy data-dependent regret bounds in convex online optimization. They are especially effective when features are sparse or heterogeneous.

5. Distinguish AdaGrad from Adam and saddle escape

AdaGrad's accumulated second moments G_t grow without forgetting; Adam (Ref. [43]) instead uses exponentially moving first and second moments and bias corrections. AdaGrad's 2011 regret guarantee does *not* certify avoidance of strict saddles in deep non-convex loss landscapes. Such avoidance requires separate assumptions and analysis (our Ref. [59]).

6. Assumptions, limitations and survey relevance

The core proofs use convex losses, bounded feasible sets or suitable regularization, and controlled subgradients. Full-matrix adaptivity can be expensive in large dimensions. The algorithm can shrink effective steps excessively after a long history of large gradients. Relevant to **RQ2** (optimizer dynamics) and **RQ4** (convex guarantee versus non-convex applications).

Do not transfer guarantees. Low *regret* in online convex prediction is not the same as finding a global minimum, a local minimum, or an ϵ-SOSP of a non-convex neural-network loss.

7. Our notes

AdaGrad is easiest to understand as a change in the geometry of an update: coordinates with many big gradients are treated differently from rarely active ones. The interesting proof is about data-adaptive regret, not about avoiding saddles. For our paper, it motivates why the optimizer itself changes training dynamics, but we must use later results for non-convex guarantees. **Original source:** https://jmlr.org/papers/v12/duchi11a.html. **Source check:** journal manuscript.